L34 — Binary Tree Traversal Using Stacks

Unit: 3 | **Chapter:** Trees | **BT Level:** BT5 | **CO:** CO2, CO3

Learning Objectives

- Implement all four binary tree traversals recursively and iteratively
- Use an explicit stack to perform inorder, preorder, and postorder traversal
- Trace traversal on a given tree step by step
- Understand Morris Traversal (O(1) space) as an advanced technique

---

1. Traversal Overview

Traversing a binary tree means visiting every node exactly once in a defined order.

```
        1
       / \
      2   3
     / \ / \
    4  5 6  7
```

Traversal	Visit Order	Output for above tree
Inorder (LNR)	Left → Node → Right	4 2 5 1 6 3 7
Preorder (NLR)	Node → Left → Right	1 2 4 5 3 6 7
Postorder (LRN)	Left → Right → Node	4 5 2 6 7 3 1
Level-order	Level by level	1 2 3 4 5 6 7

---

2. Recursive Implementations

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def inorder_recursive(root):
    if root is None:
        return []
    return inorder_recursive(root.left) + [root.data] + inorder_recursi

def preorder_recursive(root):
    if root is None:
        return []
    return [root.data] + preorder_recursive(root.left) + preorder_recur
```

```
def postorder_recursive(root):
    if root is None:
        return []
    return postorder_recursive(root.left) + postorder_recursive(root.ri
```

---

3. Iterative Traversals Using Explicit Stack

3.1 Inorder — Iterative (LNR)

Strategy: Go as far left as possible, push each node. When we can't go left, pop and visit, then go right.

```
def inorder_iterative(root):
    result = []
    stack = []
    current = root

    while current or stack:
        # Go to the leftmost node
        while current:
            stack.append(current)
            current = current.left

        # Pop node, visit it
        current = stack.pop()
        result.append(current.data)

        # Move to right subtree
        current = current.right

    return result
```

Trace on the example tree (root = 1):

Step	current	Stack	Result	Action
1	1	[1]	[]	push 1, go left
2	2	[1,2]	[]	push 2, go left
3	4	[1,2,4]	[]	push 4, go left
4	None	[1,2,4]	[]	pop 4, visit
5	None (4.right)	[1,2]	[4]	pop 2, visit
6	5 (2.right)	[1,5]	[4,2]	push 5, go left
7	None	[1,5]	[4,2]	pop 5, visit
8	None (5.right)	[1]	[4,2,5]	pop 1, visit
9	3 (1.right)	[3]	[4,2,5,1]	push 3, go left
10	6 (3.left)	[3,6]	[4,2,5,1]	push 6, go left
11	None	[3,6]	[4,2,5,1]	pop 6, visit
12	None (6.right)	[3]	[4,2,5,1,6]	pop 3, visit
13	7 (3.right)	[7]	[4,2,5,1,6,3]	push 7, go left

14	None	[7]	[4,2,5,1,6,3]	pop 7, visit
15	None	[]	[4,2,5,1,6,3,7]	Done

Output: [4, 2, 5, 1, 6, 3, 7] ✓

3.2 Preorder — Iterative (NLR)

Strategy: Visit node, then push right child before left (so left is processed first).

```
def preorder_iterative(root):
    if not root:
        return []
    result = []
    stack = [root]

    while stack:
        node = stack.pop()
        result.append(node.data)      # Visit node
        if node.right:
            stack.append(node.right) # Push right first
        if node.left:
            stack.append(node.left)  # Push left second (processed firs

    return result
```

Trace on example tree:

Step	Pop	Visited	Stack after
1	1	[1]	[3, 2]
2	2	[1,2]	[3, 5, 4]
3	4	[1,2,4]	[3, 5]
4	5	[1,2,4,5]	[3]
5	3	[1,2,4,5,3]	[7, 6]
6	6	[1,2,4,5,3,6]	[7]
7	7	[1,2,4,5,3,6,7]	[]

Output: [1, 2, 4, 5, 3, 6, 7] ✓

3.3 Postorder — Iterative (LRN)

Postorder is trickiest iteratively. Two approaches:

Method 1 — Two Stacks:

```
def postorder_iterative_two_stacks(root):
    if not root:
        return []
    result = []
    stack1 = [root]
```

```
    stack2 = []

    while stack1:
        node = stack1.pop()
        stack2.append(node)        # Push to result stack
        if node.left:
            stack1.append(node.left)
        if node.right:
            stack1.append(node.right)

    while stack2:
        result.append(stack2.pop().data)

    return result
```

Method 2 — One Stack with Prev Tracking:

```
def postorder_iterative_one_stack(root):
    result = []
    stack = []
    current = root
    prev = None

    while current or stack:
        while current:
            stack.append(current)
            current = current.left

        current = stack[-1]   # Peek

        # If right child exists and hasn't been processed yet
        if current.right and prev != current.right:
            current = current.right   # Go right
        else:
            result.append(current.data)  # Visit
            prev = current
            stack.pop()
            current = None

    return result
```

Output for example tree: [4, 5, 2, 6, 7, 3, 1] ✓

---

4. Level-Order Traversal (BFS)

```
from collections import deque

def level_order(root):
    if not root:
        return []
    result = []
    queue = deque([root])
```

```
    while queue:
        node = queue.popleft()
        result.append(node.data)
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)

    return result


def level_order_by_level(root):
    """Returns list of lists — one per level."""
    if not root:
        return []
    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)
        level = []
        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.data)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)

    return result
```

---

5. Complexity Comparison

Method	Time	Space
Recursive traversal	O(n)	O(h) — call stack
Iterative (stack)	O(n)	O(h) — explicit stack
Level-order (BFS)	O(n)	O(w) — max width
Morris Traversal	O(n)	O(1) — no extra space

h = height of tree, w = max width of tree ($\leq n/2$ for perfect tree).

Summary

- **Inorder** (LNR): go left, visit, go right. Uses a "go left until None, then pop and go right" loop.
- **Preorder** (NLR): visit, push right, push left — result is natural root-first order.

- **Postorder** (LRN): hardest iteratively — use two stacks or track previously visited node.
- **Level-order**: BFS with a queue.
- Iterative stack-based traversal avoids recursion depth limits; O(h) space in either case.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 12.1 (MIT Press, 2022)
- LeetCode #94 (Inorder), #144 (Preorder), #145 (Postorder), #102 (Level Order)